

Where Do the Experts Go?

A Validated Speed-of-Light Model for Mixture-of-Experts Decode on Memory-Constrained Consumer Hardware

Harshith Kantamneni
Independent Researcher

Abstract

Frontier-scale Mixture-of-Experts (MoE) models reach desktops only by keeping most experts in host memory, and more than thirty systems now propose policies for which experts to cache on the GPU, how to prefetch them, and where to execute misses. What is missing is a measured ceiling that every policy can be compared against. We contribute three things. **(1)** A method to record *exact* decode-time routing traces without a GPU: because a causal model’s routing of a token depends only on its prefix, teacher-forced prefill streamed one decoder layer at a time reproduces decode routing bit-for-bit (verified against reference implementations on five architectures) while reading each weight once per corpus. On a 2-vCPU, 7 GB virtual machine we trace OLMoE, Qwen3-30B-A3B, gpt-oss-20b and gpt-oss-120b over four domains. **(2)** A five-parameter bytes-over-bandwidth model of batch-1 decode that retrodicts 52 published measurements from 12 independent sources (0.58–128 tok/s, seven models, CPU-only, static CPU offload and PCIe demand fetch) with 17% median held-out error. **(3)** A speed-of-light bound for any per-layer expert placement policy, combining Belady’s optimal replacement with Jensen’s inequality, and a closed-form reuse threshold r^* that says when copying an expert to the GPU can pay off at all. The traces show strong short-range locality (46% of Qwen3’s selected experts were selected by the previous token, $7.4\times$ the uniform rate) but hot-expert sets that do not transfer across domains (Jaccard 0.11 vs. 0.14 for random sets), so static “popular-expert” pinning calibrated on other data is worse than no pinning. On PCIe 4 desktops, executing a missed expert on the CPU is cheaper than fetching it, and the llama.cpp-style static layer split leaves up to half of the attainable throughput unused, while a plain LRU cache with CPU-executed misses closes most of the gap to the bound, leaving little for predictors to win. Code, traces and the measurement set are released.

1 Introduction

A 120B-parameter MoE with 4-bit experts occupies 61 GB; a consumer GPU has 8–32 GB. Running such models locally therefore means splitting experts between GPU memory and host DRAM, and the resulting decode speed

is set almost entirely by where each routed expert’s weights live when the router selects it. The literature has responded with many mechanisms: demand fetching into an LRU cache [3], activation-aware caches and prefetchers [19, 14, 8, 23, 17], executing misses on the CPU [10, 11, 16, 7], mixed-precision or substituted experts [15, 22], and in practice, llama.cpp’s static assignment of whole layers’ experts to the CPU [4]. Recent trace studies disagree on basic questions, such as whether a predictor beats LRU [21, 13, 17], and one explicitly notes that fewer misses need not mean higher throughput [21].

We argue that the field lacks a *bound*: for a given model, hardware platform and GPU memory budget, how fast can batch-1 decode possibly be, and how close are the strategies people actually run? A bound turns policy comparisons into a fraction of what is attainable, the way the roofline model [18] does for kernels. Building one needs (i) routing traces of real models, (ii) a cost model that is *validated* rather than assumed, and (iii) an argument that no policy can beat it. This paper supplies all three, using only public data and a CPU-only virtual machine.

Contributions.

- **Exact traces without a GPU** (Section 3). Layer-streaming, teacher-forced prefill yields decode routing exactly; each weight byte is read once per corpus rather than once per token. Bit-exact agreement with reference implementations on five architectures, including MXFP4-packed and sliding-window models.
- **A validated decode model** (Section 4). Five fitted parameters, every other input taken from datasheets and model configurations. Leave-one-source-out median error 17% over 52 published points; an ablation shows which terms matter; two-socket NUMA is the dominant unmodeled effect.
- **A speed-of-light bound and r^*** (Section 5). A policy-independent lower bound on time per token, and the minimum reuse count that makes an expert worth copying to the GPU.
- **Findings** (Section 6). Strong short-range locality, weak cross-domain popularity, CPU execution of misses dominating PCIe fetch on PCIe 4, and a quantified ceiling on what predictors can add.

2 Background and Related Work

MoE decode is a data-movement problem. At batch 1, each routed expert is a pair of matrix–vector products whose cost is the bytes of its weights divided by the bandwidth of the memory it is read from. The transformer data-movement view [9] and sparsity as a first-class efficiency lever [5] frame MoE placement naturally: the question is which bytes cross which link, and how often.

Offloading systems. Demand-paging systems keep a per-layer GPU expert cache and copy misses over PCIe [3, 19, 14, 15]; prefetchers predict the next layer’s experts [8, 23, 17]. CPU-execution systems run the experts that are not resident on the host [10, 11, 16, 7]; llama.cpp exposes this as `--n-cpu-moe` and tensor overrides [4]. MoE-Lightning [2] and MoE-Lens [20] model *batched throughput*; we target single-user latency, where the constraints differ.

Traces and evaluation methodology. Two recent trace studies analyse expert caching offline [21, 13]. The first shows that replay semantics and prompt-template contamination can reverse policy rankings and reports miss rates only; the second studies kernel-managed tiering on GH200. A dense-model study predicts llama.cpp throughput from GGUF metadata, with 36% decode error on an RTX system [12]. We follow the benchmarking-rigour guidelines of Hoefler and Belli [6]: report medians and ranges, make the fitted parameters explicit, and hold out whole sources.

3 Exact Routing Traces Without a GPU

Observation. In a causal decoder, the hidden state of token t at every layer is a function of tokens $1..t$ only. The experts that decode selects for token t are therefore exactly the experts that a teacher-forced prefill over the same sequence selects at position t , up to floating-point ties. Collecting decode traces does not require running decode.

Layer streaming. Prefill makes it possible to invert the loop order. Instead of loading every layer for every token, we load *one* decoder layer, push every sequence of the corpus through it, record the router’s top- k indices, free the layer and continue. Weights are read once per corpus, activations for the whole corpus stay resident (~ 200 MB for 25k tokens), and peak memory is one layer. Safetensors shards are downloaded on demand and deleted once consumed, so disk usage stays at a few shards. Two engineering details matter. First, the MoE sub-block is token-wise, so after per-sequence attention we run it once over the concatenated corpus, touching each expert once per layer. Second, gpt-oss stores experts in MXFP4; a

120B layer would need 13 GB in fp32, so we keep the packed tensors and dequantize one expert at a time.

Exactness. A test compares our per-layer expert selections with the reference `transformers` implementation on tiny random checkpoints of Qwen3-MoE, OLMoE, DeepSeek-V2 and gpt-oss (both MXFP4-packed and bf16, with sequences longer than gpt-oss’s 128-token sliding window). All selections match exactly. For the real models we also report teacher-forced perplexity as an end-to-end check that weights were loaded correctly (Table 2).

Corpus. We build 160 two-turn conversations from four public datasets: chat (UltraChat), math (NuminaMath-CoT), code (CodeFeedback) and multilingual (Aya, non-English). Each is formatted with the model’s own chat template and nothing else, avoiding shared-template contamination [21]. We take about 6k tokens per domain per model. Prompt tokens are prefill; only assistant tokens are decode steps. Conversations are replayed back-to-back in a fixed shuffled order, as one user’s session, so cache state carries across conversations and domains.

Cost. The whole collection ran on a 2-vCPU, 7 GB RAM cloud VM with no GPU. Qwen3-30B-A3B (61 GB of bf16 weights) took about 25 minutes of compute over 25k tokens, dominated by the download.

4 A Bytes-over-Bandwidth Decode Model

Model. For one decode step,

$$T = \frac{D + X_g}{\eta_g B_g} + \frac{X_c}{\eta_c B_c} + \frac{X_f}{\eta_p B_p} + n_f \tau_x + n_h \tau_h, \quad (1)$$

where D is the per-token dense traffic: attention, shared experts, dense MLPs, routers, norms, LM head and KV-cache reads. X_g , X_c and X_f are the routed-expert bytes executed from GPU memory, executed on the CPU, and fetched over PCIe. B_g , B_c and B_p are datasheet peak bandwidths of GPU memory, host DRAM (channels $\times$ MT/s $\times$ 8 B) and the host–device link. n_f transfers each pay a fixed latency τ_x , and n_h GPU $\leftrightarrow$ CPU hand-offs (one per layer with CPU work) each pay τ_h . Byte counts are exact: we instantiate each architecture on PyTorch’s `meta` device and classify every parameter, and we take bits per weight from the quantization format or from the model file size when the source reports it.

Validation data. We collected published decode measurements with full hardware descriptions from papers, project documentation and detailed public benchmark threads (52 points from 12 groups; each row cites its source and the quoted table or line). They span CPU-only execution on 2- to 12-channel DDR5, static CPU

Table 1: Model-term ablation: median absolute percentage error (leave-one-source-out) and MAPE.

Variant	params	median	MAPE
M0 bytes/BW + hand-off	4	21%	24%
M1 + per-transfer latency	5	17%	18%
M2 + per-layer fixed costs	7	17%	19%
M3 + server/desktop η_c	8	19%	20%

offload (llama.cpp, ik_llama.cpp, KTransformers and a 2026 research engine [16]), and demand fetch over PCIe [3]. The models are gpt-oss-20b and 120b, Qwen3-30B-A3B and 235B-A22B, DeepSeek-V3/R1, Kimi-K2 and Mixtral. Rows whose RAM speed was not stated (two), approximate web-UI readings, and VM runs are flagged; two-socket runs of llama.cpp-family engines are held out entirely (below).

Fitting and held-out evaluation. We fit the efficiencies η and latencies τ by least squares on $\log(\hat{y}/y)$. We evaluate by *leave-one-source-out* cross-validation: all rows from one paper or thread are held out and predicted by a model fit on the rest. The demand-fetch rows come from a single paper, so for them we hold out one GPU at a time. Table 1 shows the ablation. Bytes over bandwidth with a hand-off term (M0) already reaches 21% median held-out error. Adding a per-transfer latency (M1) fixes the fetch rows, because the naive whole-layer copy and the per-expert copy differ in transfer count, not bytes. Per-layer fixed costs (M2) and a server/desktop CPU split (M3) do not improve held-out error. We select M1: $\eta_g = 0.39$, $\eta_c = 0.53$, $\eta_p = 0.86$, $\tau_h = 99 \mu\text{s}$ and $\tau_x = 5.8 \text{ ms}$. The fitted τ_x describes one Python prototype’s per-copy overhead: its bulk copies run at $\approx 87\%$ of PCIe peak while each expert copy costs milliseconds. For speed-of-light analysis we therefore use $\tau_x = 10 \mu\text{s}$, a DMA-engine latency.

Results. With M1, the held-out median error is 17% (MAPE 18%). 77% of points fall within 25%, and the worst is 38% (Fig. 1). By execution style, the median is 24% for CPU-only, 16% for static offload and 15% for demand fetch. Fitting a CPU efficiency per engine ranks the implementations as expected: 0.76 for the 2026 AVX-512 engine [16], 0.57 for ik_llama.cpp, 0.53 for llama.cpp and 0.42 for KTransformers’ early releases. The 8 two-socket llama.cpp-family runs are over-predicted by a median of 87%, because these engines do not scale across sockets; the model assumes one NUMA domain, and we report this as its main limitation rather than adding a fitted fudge factor.

5 A Speed-of-Light Bound

Consider any policy that keeps at most c experts per layer on the GPU. Each routed-expert execution is served in

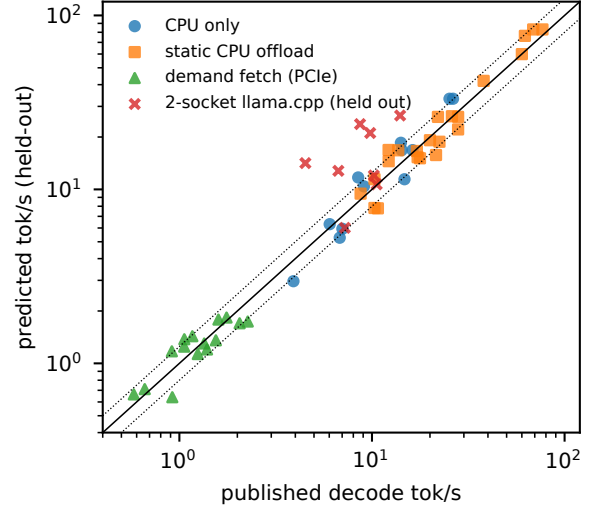


Figure 1: Held-out predictions vs. published decode throughput (log-log; dotted lines $\pm 25\%$). Crosses: two-socket llama.cpp runs, not used for fitting.

one of three ways: from GPU memory (cost $a = s/\eta_g B_g$ for an expert of s bytes), by fetching it and then running it on the GPU (cost $p + a$ with $p = s/\eta_p B_p$, serial with the layer), or on the CPU concurrently with the GPU (cost $b = s/\eta_c B_c$). With m_f fetched and m_c CPU executions out of k in a layer step, the layer takes at least

$$g(m_f, m_c) = \max((k - m_c)a + m_f p, m_c b). \quad (2)$$

g is convex, being a maximum of affine functions. **Bound.** Let M^* be the minimum total number of misses any policy can incur on the trace with per-layer capacity c . If misses may be served without admission, this is Belady’s MIN with bypass [1]; we verify its optimality against exhaustive search on random small instances. Every policy has mean misses $\bar{m} = \bar{m}_f + \bar{m}_c \geq M^*/N$ over N layer steps, so by Jensen’s inequality the mean layer time is at least

$$\min_{m_f + m_c \geq M^*/N} g(m_f, m_c). \quad (3)$$

Adding the dense time gives a lower bound on time per token for every policy in the class, whether static or dynamic, predictive or not. Admission traffic, hand-offs and transfer latency only add time, so dropping them keeps the bound valid.

When is copying an expert worth it? Copying an expert costs p once. Afterwards, each GPU execution saves $b - a$ relative to running it on the CPU. The copy pays off only if the expert is reused at least

$$r^* = \frac{p}{b - a} = \frac{1/(\eta_p B_p)}{1/(\eta_c B_c) - 1/(\eta_g B_g)} \quad (4)$$

times while resident. r^* does not depend on the model. It is 6.4 for an RTX 4060 on PCIe 4 $\times$ 8 with DDR5-5600,

Table 2: Routing statistics over decode tokens (per-layer averages). Reuse@ w : fraction of selections whose expert was selected within the previous w tokens. Top-25%: share of selections taken by the most popular quarter of experts. Jaccard: overlap of different domains’ top-25% sets (random: 0.14). Hit rates are at a GPU cache of 25% of each layer’s experts.

	OLMoE	Qwen3-30B	gpt-oss-20b	gpt-oss-120b
E / k	64/8	128/8	??/??	??/??
decode tokens	20,115	20,480	??	??
teacher-forced ppl	3.67	5.43	??	??
reuse@1 (uniform)	46% (12.5)	46% (6.2)	??% (??)	??% (??)
reuse@16	90%	88%	??%	??%
top-25% share	49%	49%	??%	??%
domain Jaccard	0.19	0.11	??	??
hit: static layers	25%	25%	??%	??%
hit: hot, held-out dom.	32%	14%	??%	??%
hit: hot, oracle	49%	49%	??%	??%
hit: LRU	65%	82%	??%	??%
hit: Belady	77%	91%	??%	??%
hit: Belady-bypass	80%	91%	??%	??%

2.2 for an RTX 4090 on PCIe 4 with DDR5-6000, and 1.1 for an RTX 5090 on PCIe 5 with DDR5-6400. On weak GPUs, the GPU is barely faster than the CPU for expert GEMV, so almost nothing is worth copying. On PCIe 5 with a fast GPU, one reuse suffices. For the same reason, fetching a miss on the critical path ($p + a$) loses to executing it on the CPU (b) whenever $\eta_p B_p < \eta_c B_c$, which holds on every PCIe 4 desktop we consider.

6 Results

6.1 Routing locality

Table 2 and Fig. 2 summarise routing behaviour. Short-range reuse is strong in every model. For Qwen3-30B, 46% of selections repeat an expert chosen by the *previous* token, $7.4\times$ the uniform-routing rate, and 88% repeat within 16 tokens. A 25% LRU cache consequently hits 82%, within a few points of Belady (91%). Long-range popularity is weaker, and it does not transfer. The most popular quarter of experts takes only about half of all selections, and the popular sets of different domains overlap *less* than random sets would (Jaccard 0.11 vs. 0.14 for Qwen3). Pinning the experts that are hottest on the other three domains therefore hits only 14% on the held-out domain, fewer than simply pinning 25% of the layers. The oracle version, calibrated on the same data it is evaluated on, reaches 49%, which shows how much in-distribution calibration flatters static pinning.

Table 3: Decode tok/s at each platform’s VRAM-matched operating point (experts that fit after dense weights, 8k-token KV cache and 1.5 GB reserve), with the fitted hand-off latency. “% SoL” is relative to the speed-of-light bound.

Model	Strategy	4060 8GB	4090	5090
1* gpt-oss-120b	GPU cache fraction	??%	??%	??%
	static layers (llama.cpp)	??	??	??
	% SoL	??%	??%	??%
	LRU, fetch on miss	??	??	??
	LRU, CPU misses	??	??	??
Qwen3-30B	% SoL	??%	??%	??%
	speed of light	??	??	??
	GPU cache fraction	27%	100%	100%
	static layers	23.8	124.9	222.1
	LRU, CPU misses	32.7	124.6	220.7
gpt-oss-120b	speed of light	37.5	129.8	227.5

6.2 Throughput against the bound

Figure 3 and Table 3 convert hit rates into throughput with the validated model. Four observations follow.

(i) **Where misses execute matters more than the replacement policy.** On PCIe 4 desktops, the same LRU cache is faster with CPU-executed misses than with fetch-on-miss at every cache size (Fig. 3), because $\eta_p B_p < \eta_c B_c$ (Section 5). This explains why desktop users converge on llama.cpp’s CPU offload rather than on fetch-based research systems. On PCIe 5 the two approach parity.

(ii) **The static layer split leaves throughput unused.** At the VRAM-matched point, llama.cpp-style layer assignment reaches ??% of the bound for gpt-oss-120b on an RTX 4090 and ??% on an RTX 5090. Placing the same bytes as a per-layer LRU cache with CPU-executed misses reaches ??% and ??%, a $??\times$ and $??\times$ speedup over the static split.

(iii) **The headroom left for predictors is bounded.** Once misses run on the CPU, the difference between LRU and Belady-bypass in hit rate translates into a much smaller difference in tok/s. The remaining gap to the bound (Table 3) caps what *any* prefetcher or learned predictor can add on these platforms. Much of that remaining gap comes from CPU–GPU concurrency that the bound allows, not from misses.

(iv) **Weak GPUs gain little from any policy.** On the RTX 4060 the GPU’s expert bandwidth is close to the CPU’s ($r^*=6.4$), so all strategies cluster near CPU-only speed. The lever there is DRAM bandwidth, not caching.

7 Discussion

For engine builders. The bound says the first-order win on desktops is a *dynamic* expert cache with CPU-executed misses and asynchronous admission, not better static placement or prediction. llama.cpp has no dynamic GPU expert cache today; community prototypes report

large gains, consistent with our model. Admission should respect r^* : on PCIe 4 an expert should be copied only if it is expected to be reused a few times.

For evaluation. Hit rate is a poor proxy for speed when misses can execute elsewhere. Policies should be reported as a fraction of a hardware-specific bound, with the hand-off and admission costs stated. Our measurement set and model make that cheap.

For model design. Equation (1) and the traces also run in reverse. Given a target platform (B_g, B_c, B_p , VRAM), they predict decode speed as a function of expert count, expert size, top- k and shared-expert size, with locality measured rather than assumed. This points to designing MoE architectures for a consumer memory hierarchy: for example, choosing expert granularity so that r^* -worthy reuse is common. We leave this co-design, which requires training models, to future work.

8 Limitations

Off-policy text. Traces are teacher-forced on dataset responses, not on each model’s own samples. Routing is exact for the given text, but self-generated text may be more predictable. **Batch 1 only;** small-batch serving unions experts across requests and weakens locality. **The model has no NUMA term** and over-predicts two-socket llama.cpp; it also has no explicit CPU core-count limit, and fitted efficiencies absorb implementation quality. **Validation is retrodictive:** the author had no hardware, so all measurements are third-party. Some are approximate or from VMs (flagged in the data), and two assume unstated DDR5 speeds. **Per-layer capacity:** the bound covers policies that partition the cache per layer; a global pool shared across layers could do better. **Numerics:** traces were computed in fp32 on CPU; GPU inference in bf16 may break near-ties differently.

9 Conclusion

A causal model’s routing is fixed by its prefix, so exact decode traces of large MoEs can be collected on a small CPU machine. A five-parameter bytes-over-bandwidth model, fitted and held out across a dozen independent sources, predicts batch-1 decode within about a sixth. Combined with Belady and Jensen, these give a speed-of-light bound for consumer MoE inference. The bound shows that on today’s desktops, where the misses execute matters more than how cleverly they are predicted.

Artifact. Code, traces, the annotated measurement set and scripts that regenerate every number and figure are released with this paper.

Acknowledgments. Portions of the code, the literature search and the text were drafted with the assistance of an AI system (Claude, Anthropic). The author reviewed the work and takes responsibility for its content.

References

- [1] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [2] Shiyi Cao, Shu Liu, et al. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. *arXiv preprint arXiv:2411.11217*, 2024.
- [3] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- [4] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023–2026.
- [5] Torsten Hoefer, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241), 2021.
- [6] Torsten Hoefer and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [7] En-Ming Huang, Li-Shang Lin, et al. Efficient CPU-GPU collaborative inference for MoE-based LLMs on memory-limited systems. *arXiv preprint arXiv:2512.16473*, 2025.
- [8] Ranggi Hwang, Jianyu Wei, et al. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. *arXiv preprint arXiv:2308.12066*, 2023.
- [9] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefer. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- [10] Keisuke Kamahori, Tian Tang, et al. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033*, 2024.
- [11] KVCache.AI and contributors. KTransformers: A flexible framework for CPU/GPU hybrid LLM inference. <https://github.com/kvcache-ai/ktransformers>, 2024–2026. See also the SOSP 2025 paper by the same team.
- [12] Xinyu Qiu, Chuhong Xu, et al. GGUF-metadata prediction of single-sequence llama.cpp throughput across three systems. *arXiv preprint arXiv:2609.14864*, 2026.
- [13] Yuan Si, Yufeng Lin, et al. Who should own the expert cache? kernel-managed tiering for trillion-parameter MoE inference. *arXiv preprint arXiv:2608.12103*, 2026.
- [14] Xiaoni Song, Zihang Zhong, et al. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024.

- [15] Peng Tang, Jiacheng Liu, et al. HOBBIT: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433*, 2024.
- [16] Wenxin Wang, Yule Hou, et al. Achieving cloud-grade SLOs for local mixture-of-experts inference through CPU-GPU hybrid design. *arXiv preprint arXiv:2606.10493*, 2026.
- [17] Zihan Wang, Yuqi Wang, et al. SeqMoE: Toward full-load performance via predictive and graph-compatible MoE offloading. *arXiv preprint arXiv:2609.12978*, 2026.
- [18] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [19] Leyang Xue, Yao Fu, et al. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.
- [20] Yichao Yuan, Lin Ma, et al. MoE-Lens: Towards the hardware limit of high-throughput MoE LLM serving under resource constraints. *arXiv preprint arXiv:2504.09345*, 2025.
- [21] Yu Zhang. Reproducible evaluation of MoE expert caching: Replay semantics, workload contamination, and operating regimes. *arXiv preprint arXiv:2608.07911*, 2026.
- [22] Yushu Zhao, Yubin Qin, et al. MoBiLE: Efficient mixture-of-experts inference on consumer GPU with mixture of big little experts. *arXiv preprint arXiv:2510.12357*, 2025.
- [23] Shien Zhu, Samuel Bohl, et al. Pre-attention expert prediction and prefetching for mixture-of-experts large language models. *arXiv preprint arXiv:2511.10676*, 2025.

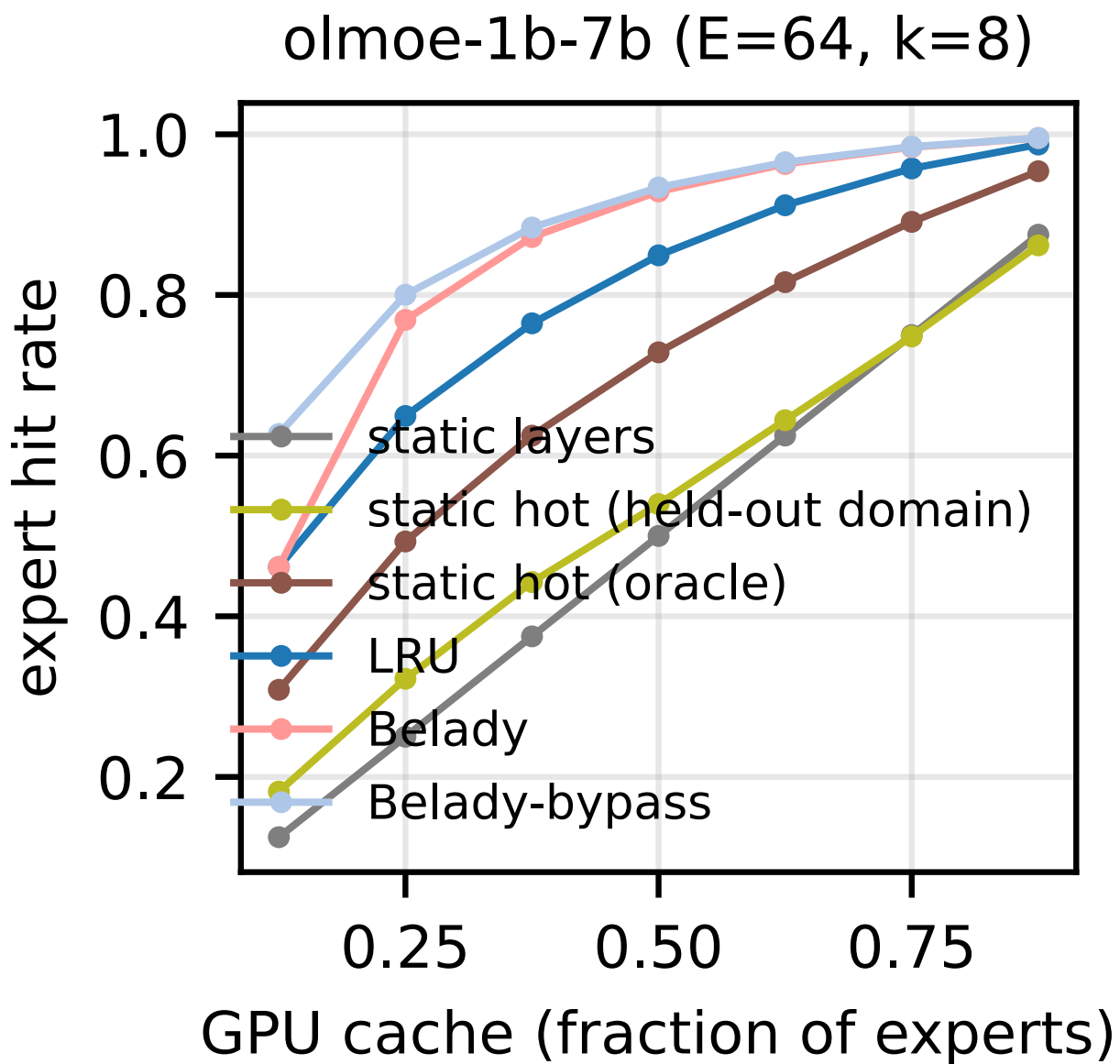


Figure 2: Expert hit rate vs. GPU cache size (fraction of each layer’s experts) for static and dynamic policies.

figs/toks_gpt-oss-120b.pdf

Figure 3: gpt-oss-120b decode tok/s vs. GPU cache size on three desktop platforms (validated model, hand-off 20 μ s). Dashed: speed-of-light bound for any per-layer placement policy. Dotted grey: all experts on the CPU.